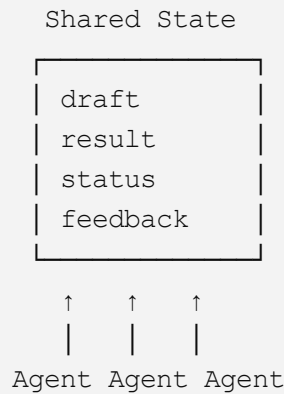


Agent 간 메시지 전달 구조

1. Agent 간 메시지 전달의 개념

멀티에이전트 시스템에서는 여러 Agent가 서로 작업 결과를 전달하면서 하나의 문제를 해결한다.

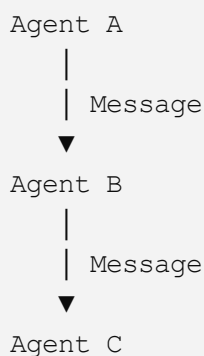
가장 단순한 방식은 모든 Agent가 하나의 공유 State를 함께 읽고 수정하는 방식이다.



이 방식은 구현하기 쉽다는 장점이 있지만 Agent가 많아지면 문제가 발생할 수 있다.

- 어떤 Agent가 특정 데이터를 변경했는지 추적하기 어렵다.
- 모든 Agent가 공유 State의 구조를 알아야 한다.
- Agent 간 의존성이 커질 수 있다.
- Agent를 독립적인 서비스로 분리하기 어렵다.

메시지 전달(Message Passing) 방식에서는 공유 State를 직접 수정하는 대신 Agent가 다른 Agent에게 메시지를 전달한다.



첨부 실습에서는 이러한 차이를 명확하게 보여주기 위해 주문 처리 시스템을 예제로 사용하고 있으며, `order_intake` → `inventory` → `shipping` → `customer_service` Agent가 메시지만 주고 받는 구조로 구성되어 있다.

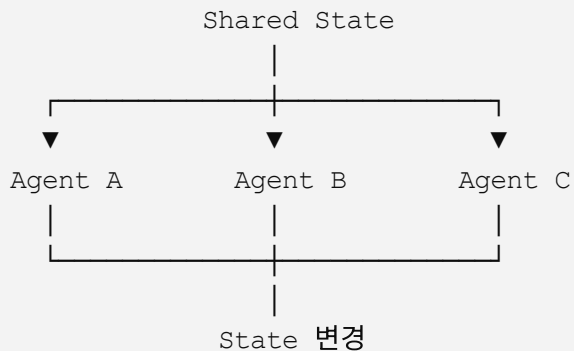
03_Agent 간 메시지 전달 구조

2. Shared State 방식과 Message Passing 방식

두 방식의 차이를 먼저 이해할 필요가 있다.

Shared State 방식

모든 Agent가 하나의 State를 공유한다.



예를 들어:

Python

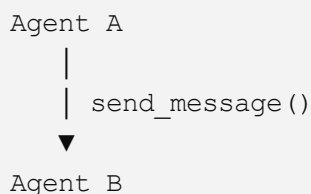
실행됨

```
state["result"] = "재고 있음"
```

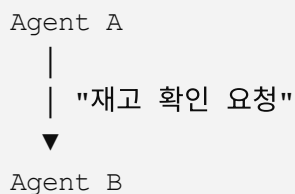
Agent가 직접 공유 State를 변경한다.

Message Passing 방식

Agent가 다른 Agent에게 메시지를 보낸다.



Agent B는 Agent A의 내부 State를 알 필요가 없다.



Agent B는 전달받은 메시지만 처리한다.

3. Message Passing의 핵심 개념

Message Passing 구조에서는 Agent 간 통신을 명시적인 Message 객체로 표현한다.

기본적인 메시지는 다음 정보를 포함할 수 있다.

```
Message
├── sender
├── receiver
├── type
└── content
```

첨부 실습에서도 동일한 구조의 `Message` 모델을 사용한다. `sender` 는 메시지를 보낸 Agent, `receiver` 는 받을 Agent, `type` 은 메시지 종류, `content` 는 실제 내용을 의미한다.

03_Agent 간 메시지 전달 구조

예를 들어:

Python

실행됨

```
Message (
    sender="order_intake",
    receiver="inventory",
    type="request",
    content="무선 키보드 2개 재고 확인 요청"
)
```

메시지를 문자열 하나로 전달하는 것보다 구조화된 객체로 전달하면 시스템에서 메시지를 분석하고 라우팅하기 쉬워진다.

4. Message의 구성 요소

4.1 sender

메시지를 보낸 Agent이다.

```
sender = "order_intake"
```

즉, 주문 접수 Agent가 메시지를 보냈다는 의미이다.

4.2 receiver

메시지를 받을 Agent이다.

```
receiver = "inventory"
```

따라서 해당 메시지는 재고 확인 Agent에게 전달된다.

4.3 type

메시지의 목적을 나타낸다.

기본적으로 다음 세 가지 유형을 사용할 수 있다.

```
request
response
notify
```

request

작업을 요청하는 메시지이다.

```
order_intake
  |
  | request
  ▼
inventory
```

response

요청에 대한 결과를 반환하는 메시지이다.

```
inventory
  |
  | response
  ▼
order_intake
```

notify

특정 Agent에게 결과를 통보하는 메시지이다.

```
order_intake
  |
  | notify
  ▼
customer_service
```

첨부 실습에서도 `request`, `response`, `notify` 세 가지 메시지 유형을 정의한다.

03_Agent 간 메시지 전달 구조

5. Message 구조 예시

주문 처리 시스템을 예로 들어보자.

사용자가 무선 키보드 2개를 주문했다고 가정한다.

첫 번째 메시지는 다음과 같다.

```
sender    : order_intake
receiver  : inventory
type      : request
content   : 무선 키보드 2개 재고 확인 요청
```

이를 흐름으로 표현하면 다음과 같다.

```
order_intake --request--> inventory
              재고 확인 요청
```

재고 Agent가 재고를 확인한 후 결과를 반환한다.

```
order_intake <--response-- inventory
              재고 가능
```

이후 주문 접수 Agent는 배송 Agent에게 요청한다.

```
order_intake --request--> shipping
              배송 준비 요청
```

배송 Agent가 운송장 정보를 반환한다.

```
order_intake <--response-- shipping
              운송장 정보
```

마지막으로 고객 응대 Agent에게 결과를 통보한다.

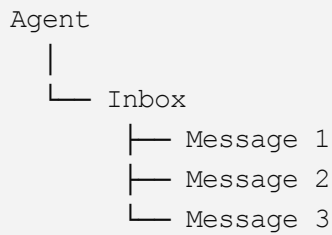
```
order_intake --notify--> customer_service
              처리 결과 통보
```

첨부된 실습의 전체 메시지 흐름도 이러한 구조로 구성되어 있다.

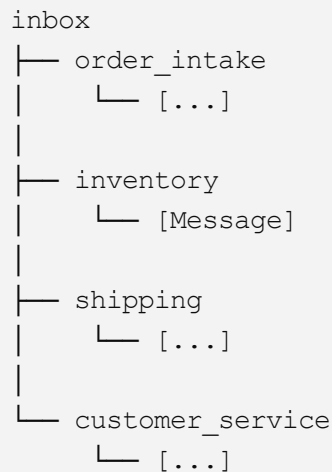
03_Agent 간 메시지 전달 구조

6. Inbox 구조

Message Passing 방식에서는 각 Agent가 자신에게 전달된 메시지를 확인할 수 있도록 Inbox를 둔다.



전체 시스템에서는 다음과 같이 관리할 수 있다.



첨부 실습의 State에서도 Agent별 수신 대기 메시지를 저장하는 `inbox` 를 `Dict[str, List[dict]]` 형태로 정의한다.

03_Agent 간 메시지 전달 구조

7. Agent의 기본 처리 패턴

Message Passing 방식에서 각 Agent는 거의 동일한 패턴으로 동작한다.

- ① Inbox 확인
- ↓
- ② 메시지 읽기
- ↓
- ③ 작업 수행
- ↓
- ④ 결과를 Message로 생성
- ↓
- ⑤ 다른 Agent에게 전송
- ↓
- ⑥ 다음 Agent 지정

이를 코드 관점에서 표현하면 다음과 같다.

Python

실행됨

```
def agent_node(state):
    # 1. Inbox에서 자신의 메시지 확인
    message = ...

    # 2. 작업 수행
    result = ...

    # 3. 다른 Agent에게 메시지 전달
    log, inbox = send_message(
        ...,
        sender="agent_a",
        receiver="agent_b",
        mtype="response",
        content=result
    )

    # 4. 다음 실행 Agent 지정
    return {
        "log": log,
        "inbox": inbox,
        "next_agent": "agent_b"
    }
```

첨부 실습에서도 각 Agent는 자신의 Inbox에서 메시지를 읽고, 작업을 수행한 다음

`send_message` 를 통해 결과를 전달하는 동일한 패턴을 사용한다.

03_Agent 간 메시지 전달 구조

8. send_message 함수

Agent가 직접 다른 Agent의 State를 변경하지 않고 메시지 전송 함수를 사용하도록 설계할 수 있다.

Python

실행됨

```
def send_message(
    log,
    inbox,
    sender,
    receiver,
    mtype,
    content
):
    msg = Message(
        sender=sender,
        receiver=receiver,
        type=mtype,
        content=content
```

```

).model_dump()

new_log = log + [msg]

new_inbox = {
    agent: list(msgs)
    for agent, msgs in inbox.items()
}

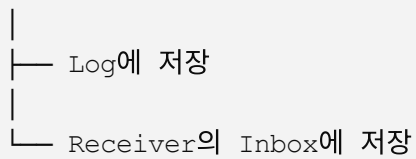
new_inbox.setdefault(receiver, []).append(msg)

return new_log, new_inbox

```

이 함수는 크게 두 가지 일을 수행한다.

Message 생성



첨부 실습에서도 `send_message` 가 메시지를 생성한 후 전체 메시지 로그와 수신자의 Inbox에 각각 저장하도록 구현되어 있다.

03_Agent 간 메시지 전달 구조

9. Message Log

Message Passing 구조에서는 메시지를 모두 Log에 기록하는 것이 중요하다.

log

1. order_intake → inventory
request
재고 확인 요청
2. inventory → order_intake
response
재고 가능
3. order_intake → shipping
request
배송 준비 요청
4. shipping → order_intake
response
운송장 발급
5. order_intake → customer_service
notify

이러한 Log를 이용하면 전체 Agent의 통신 과정을 추적할 수 있다.

10. Audit Trail

Message Log의 중요한 장점은 Audit Trail을 만들 수 있다는 것이다.

Audit Trail은 시스템에서 어떤 작업이 언제 어떤 주체에 의해 수행되었는지 추적할 수 있는 기록이다.

예를 들어:

```
10:01:03
order_intake → inventory
재고 확인 요청

10:01:04
inventory → order_intake
재고 가능

10:01:04
order_intake → shipping
배송 준비 요청

10:01:05
shipping → order_intake
운송장 발급

10:01:05
order_intake → customer_service
주문 처리 완료
```

따라서 문제가 발생했을 때 다음과 같은 질문에 답할 수 있다.

누가 보냈는가?
누구에게 보냈는가?
무슨 메시지였는가?
어떤 유형이었는가?
어떤 순서로 처리되었는가?

첨부 실습에서도 모든 메시지를 `log`에 기록하여 전체 통신 과정을 추적할 수 있도록 설계한다.

03_Agent 간 메시지 전달 구조

11. Message 기반 Routing

Message Passing 구조에서 중요한 개념은 Receiver가 다음 실행 Agent를 결정한다는 것이다.

예를 들어:

Message

```
sender    = order_intake
receiver  = inventory
```

그러면 Router는 `inventory` 를 확인하고 다음 Node를 실행한다.

```
receiver
  ↓
inventory
  ↓
inventory Agent 실행
```

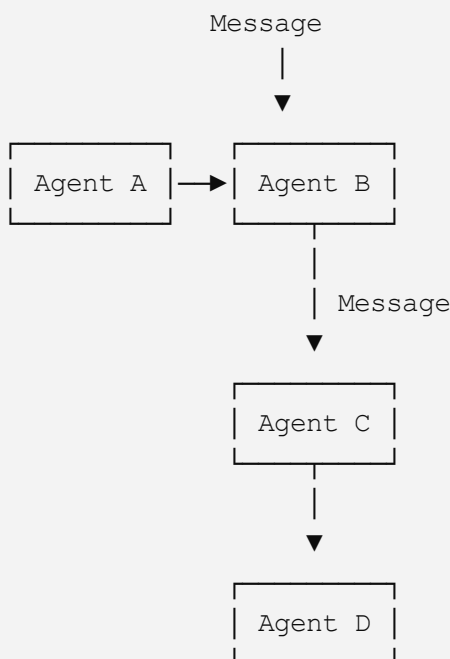
따라서 Router가 주문 처리 업무의 내용을 직접 알 필요가 없다.

첨부 실습에서는 라우팅 규칙을 매우 단순하게 구성하여 `next_agent` , 즉 마지막 메시지의 Receiver를 다음 실행 노드로 사용한다.

03_Agent 간 메시지 전달 구조

12. Message-driven Architecture

이 구조를 일반화하면 다음과 같은 형태가 된다.



Agent 간의 결합은 직접적인 State 공유가 아니라 Message Protocol을 중심으로 이루어진다.

즉,

```
Agent A
  ↓
Message Protocol
  ↓
Agent B
```

구조가 된다.

13. Agent 간 결합도 감소

Shared State 구조에서는 Agent가 State의 구조를 알아야 한다.

```
Agent A
  |
  | State 직접 접근
  ▼
Shared State
  ▲
  |
Agent B
```

따라서 Agent 간 결합도가 높아질 수 있다.

Message Passing에서는 다음과 같이 변경된다.

```
Agent A
  |
  | Message
  ▼
Agent B
```

Agent A는 Agent B의 내부 구현을 알 필요가 없다.

다음 정보만 알면 된다.

```
receiver
message type
message format
content
```

이것이 Message Passing 구조의 중요한 장점이다.

14. Agent 추가가 쉬워지는 구조

예를 들어 기존 시스템이 다음과 같다고 가정한다.

```
order_intake
  ↓
inventory
  ↓
shipping
  ↓
customer_service
```

여기에 결제 Agent를 추가할 수 있다.

```
order_intake
  ↓
inventory
  ↓
payment
  ↓
shipping
  ↓
customer_service
```

기존 Agent의 내부 로직을 크게 변경하지 않고 새로운 Message의 Receiver를 `payment` 로 지정하는 방식으로 확장할 수 있다.

첨부 실습에서도 라우터는 특정 업무의 의미를 판단하지 않고 메시지의 Receiver만 확인하므로 Agent를 추가하거나 교체하기 쉬운 구조를 강조한다.

03_Agent 간 메시지 전달 구조

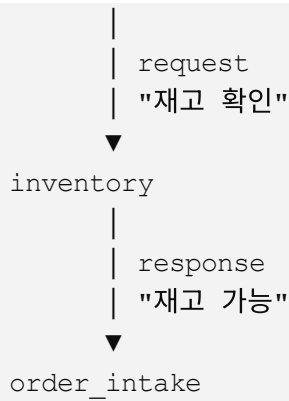
15. Request-Response 패턴

가장 기본적인 Agent 통신 패턴은 Request-Response이다.



예:

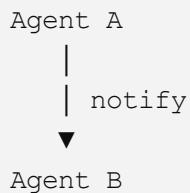
```
order_intake
```



이 구조는 데이터 조회, 계산, 분석 요청 등에 적합하다.

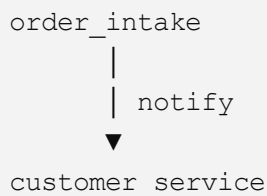
16. Notification 패턴

Notification은 결과를 단순히 통보하는 방식이다.



Response와 달리 반드시 답변을 요구하지 않는다.

예:



내용:

"주문이 정상적으로 처리되었습니다."

Customer Service는 이 정보를 이용해 고객에게 자연스러운 안내문을 생성할 수 있다.

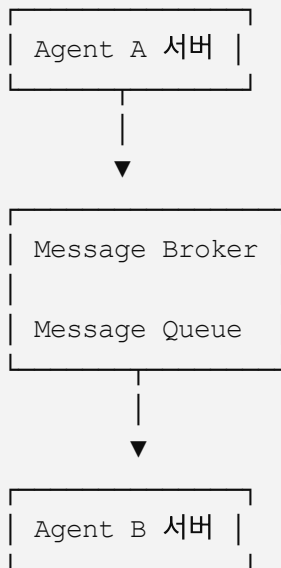
첨부 실습에서도 고객 응대 Agent는 내부 Agent가 전달한 메시지를 LLM을 이용하여 고객에게 전달할 문장으로 변환하는 종착점 역할을 한다.

03_Agent 간 메시지 전달 구조

17. 실제 서비스 환경으로의 확장

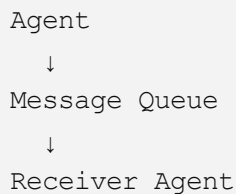
Notebook에서는 State 안에 Inbox를 구현하지만 실제 서비스에서는 Agent가 서로 다른 프로세스나 서버에서 실행될 수 있다.

개념적으로 다음과 같이 확장할 수 있다.



이 경우 Agent는 직접 상대방의 메모리나 State를 수정하지 않는다.

대신 다음과 같은 통신 방식을 사용할 수 있다.



메시지 큐를 이용하면 Agent를 독립적으로 배포하고 확장할 수 있다.

첨부 자료에서도 이러한 구조를 실제 서비스에서 메시지 큐 기반 Agent 간 통신으로 확장할 수 있는 개념으로 설명한다.

03_Agent 간 메시지 전달 구조

18. Message Schema의 중요성

Agent가 많아질수록 Message의 형식을 명확하게 정의해야 한다.

예를 들어 다음과 같은 표준을 사용할 수 있다.

Python

```
class Message(BaseModel):  
    sender: str  
    receiver: str
```

실행됨

```

type: Literal[
    "request",
    "response",
    "notify"
]
content: str

```

더 복잡한 시스템에서는 다음과 같은 필드를 추가할 수 있다.

```

Message
├─ message_id
├─ sender
├─ receiver
├─ type
├─ timestamp
├─ correlation_id
├─ priority
├─ content
└─ metadata

```

각 필드의 역할은 다음과 같다.

필드	역할
message_id	메시지 고유 식별자
sender	발신 Agent
receiver	수신 Agent
type	메시지 유형
timestamp	생성 시간
correlation_id	요청과 응답 연결
priority	처리 우선순위
content	메시지 내용
metadata	추가 정보

단순 실습에서는 `sender`, `receiver`, `type`, `content` 만으로도 Message Passing의 핵심 원리를 구현할 수 있다.

19. Correlation ID

실제 시스템에서는 하나의 요청에 여러 메시지가 발생할 수 있다.

예를 들어 주문번호 `ORD-1001` 에 대해 다음과 같은 메시지가 발생할 수 있다.

```
ORD-1001
├── 재고 확인
├── 결제 확인
├── 배송 요청
└── 고객 안내
```

따라서 각 메시지가 어느 요청에 속하는지 구분할 필요가 있다.

이때 `correlation_id` 를 사용할 수 있다.

```
Message

message_id:
MSG-001

correlation_id:
ORD-1001

sender:
order_intake

receiver:
inventory
```

이렇게 하면 하나의 사용자 요청에서 발생한 여러 Agent 간 메시지를 묶어서 추적할 수 있다.

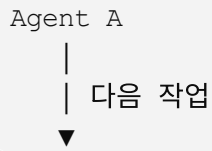
20. 동기식과 비동기식 메시지 전달

Agent 간 메시지 전달은 크게 동기식과 비동기식으로 생각할 수 있다.

동기식

요청을 보낸 Agent가 응답을 기다린다.

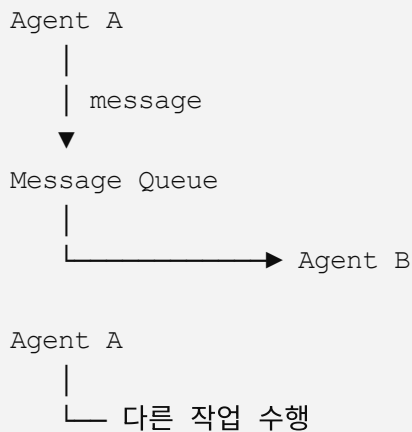
```
Agent A
|
| request
▼
Agent B
|
| response
▼
```

구조가 단순하고 결과를 즉시 사용해야 하는 작업에 적합하다.

비동기식

메시지를 보내고 응답을 기다리지 않고 다른 작업을 수행할 수 있다.



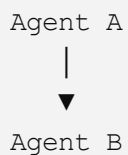
대규모 시스템이나 장시간 작업에서는 비동기 구조가 유리할 수 있다.

21. Broadcast와 Point-to-Point

Agent 간 메시지 전달 방식은 수신 대상에 따라서도 구분할 수 있다.

Point-to-Point

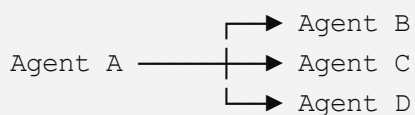
특정 Agent 하나에게 메시지를 전달한다.



주문 처리 시스템의 재고 확인 요청 등이 이에 해당한다.

Broadcast

하나의 메시지를 여러 Agent에게 전달한다.



예를 들어 긴급 장애 발생 사실을 모든 관련 Agent에게 통보할 수 있다.

22. Message Passing의 장점

22.1 Agent 간 결합도 감소

Agent가 상대방의 내부 State를 직접 알 필요가 없다.

22.2 추적성 향상

모든 메시지를 Log로 남길 수 있다.

22.3 확장성 향상

새로운 Agent를 추가하기 쉽다.

22.4 독립적인 Agent 구현

Agent별로 내부 구현을 독립적으로 변경할 수 있다.

22.5 서비스 분리 가능

각 Agent를 별도의 프로세스 또는 서버로 분리할 수 있다.

22.6 명확한 통신 프로토콜

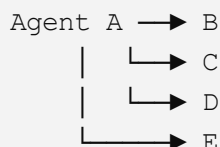
Agent 간 통신을 Message Schema로 표준화할 수 있다.

23. Message Passing의 한계

장점만 있는 것은 아니다.

메시지 관리가 복잡해질 수 있다

Agent 수가 증가하면 메시지 수가 급격히 증가할 수 있다.



메시지 순서 문제가 발생할 수 있다

비동기 시스템에서는 메시지가 예상한 순서와 다르게 도착할 수 있다.

중복 메시지를 처리해야 한다

동일한 메시지가 두 번 전달되는 상황을 고려해야 한다.

실패 처리가 필요하다

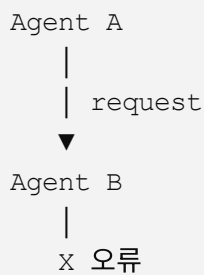
수신 Agent가 동작하지 않을 경우 재시도나 오류 처리가 필요하다.

Protocol 관리가 필요하다

Agent 간 메시지 형식이 변경되면 호환성 문제가 발생할 수 있다.

24. 오류 처리

Message Passing 구조에서는 메시지 전달 실패도 고려해야 한다.



이 경우 다음과 같은 처리 방법을 고려할 수 있다.



예:



일정 횟수 이상 실패하면:

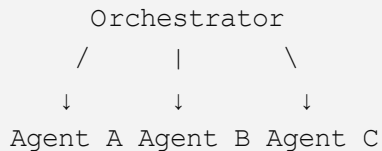
```
inventory
  ↓
error
  ↓
customer_service
```

와 같이 고객 안내 Agent로 전달할 수도 있다.

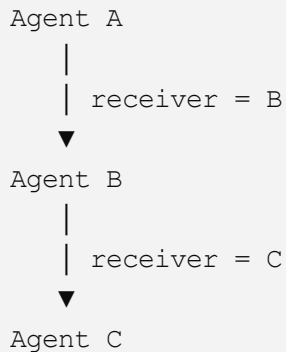
25. Message Passing과 Orchestrator의 차이

앞서 학습한 Orchestrator 구조와 Message Passing은 서로 대체 관계가 아니다.

Orchestrator 구조에서는 중앙 Agent가 다음 작업을 결정할 수 있다.



반면 Message Passing 구조에서는 Agent가 메시지를 보내면서 다음 Agent를 결정하는 구조를 만들 수 있다.



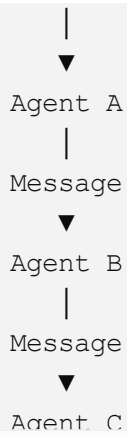
즉,

Orchestrator
→ 중앙에서 작업 흐름을 결정

Message Passing
→ 메시지의 송수신을 중심으로 Agent를 연결

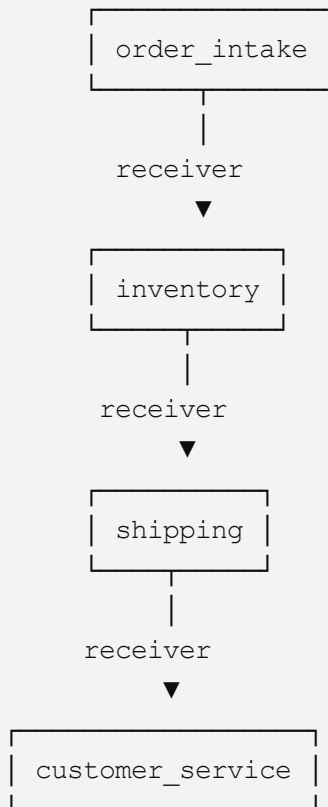
두 방식을 결합하는 것도 가능하다.

Orchestrator



26. LangGraph에서의 구현 구조

LangGraph에서는 각각의 Agent를 Node로 구성할 수 있다.



라우터는 업무의 내용을 판단하지 않는다.

Python

실행됨

```
def route_by_message(state):  
    return state["next_agent"]
```

즉,

메시지 생성

↓
receiver 확인
↓
next_agent 설정
↓
해당 Agent 실행

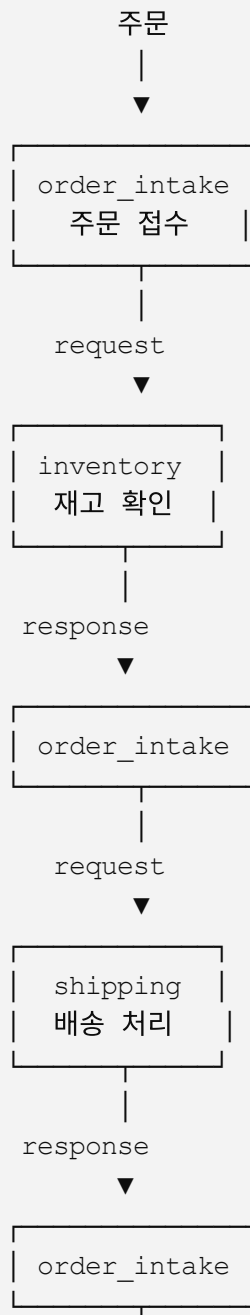
이라는 단순한 구조이다.

첨부 실습의 LangGraph 구현도 `route_by_message()` 가 `next_agent` 를 반환하고, 각 Agent 의 메시지 Receiver에 따라 다음 Node가 결정되는 방식이다.

03_Agent 간 메시지 전달 구조

27. 전체 실습 구조

첨부된 주문 처리 예제를 전체적으로 정리하면 다음과 같다.



notify

customer_service
고객 응대

각 Agent는 상대 Agent의 내부 구현을 직접 호출하지 않고 Message를 통해 협력한다.

03_Agent 간 메시지 전달 구조

28. Shared State와 Message Passing 비교

항목	Shared State	Message Passing
통신 방식	공용 State 공유	Message 전달
Agent 결합도	높아질 수 있음	낮출 수 있음
구현 난이도	낮음	상대적으로 높음
상태 추적	복잡해질 수 있음	Message Log로 추적 가능
Protocol	상대적으로 약함	명시적인 Message Schema
Agent 독립성	낮음	높음
서비스 분리	어려울 수 있음	적합
확장성	Agent 증가 시 복잡	상대적으로 유리
감사 추적	별도 구현 필요	Message Log 활용 가능
대규모 시스템	제한 가능	적합

29. 설계 원칙

Message Passing 기반 멀티에이전트 시스템을 설계할 때는 다음 원칙을 적용하는 것이 좋다.

원칙 1. Agent 간 직접 State 접근을 최소화한다

Agent A —X→ Agent B State

대신:



구조를 사용한다.

원칙 2. Message Schema를 표준화한다

모든 Agent가 동일한 Message 형식을 사용하도록 한다.

원칙 3. Receiver를 명확하게 정의한다

메시지가 어느 Agent로 전달되는지 명확해야 한다.

원칙 4. 모든 메시지를 기록한다

문제 발생 시 전체 실행 과정을 추적할 수 있도록 한다.

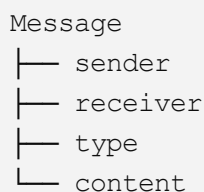
원칙 5. 메시지와 업무 로직을 분리한다

Router가 특정 업무의 내용을 직접 판단하지 않고 Message의 Receiver를 기반으로 이동하도록 설계하면 확장성이 높아진다.

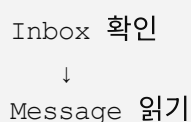
30. 핵심 정리

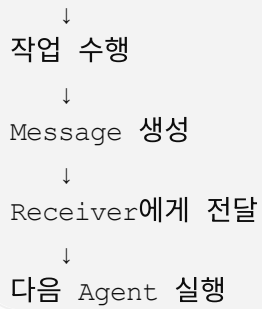
Agent 간 메시지 전달 구조는 여러 Agent가 공유 State를 직접 수정하는 대신 명시적인 Message를 통해 협력하는 구조이다.

핵심 구성은 다음과 같다.

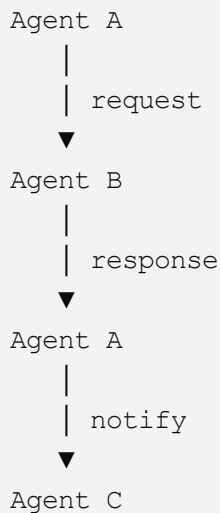


Agent의 기본 동작은 다음과 같다.





전체 시스템은 다음과 같이 구성할 수 있다.



가장 중요한 개념은 Agent가 상대 Agent의 내부 구현이나 State를 직접 알 필요 없이 "정해진 메시지 프로토콜"만 알고 통신한다는 것이다.

따라서 Message Passing 구조는 Agent 수가 증가하거나 Agent를 독립적인 서비스로 분리해야 하는 상황에서 특히 유용하다.

또한 모든 메시지를 Log로 관리하면 Agent 간 통신 과정을 추적할 수 있어 디버깅과 감사 추적에도 유리하다. 첨부 실습에서는 이러한 원리를 `order_intake`, `inventory`, `shipping`, `customer_service` 4개 Agent의 주문 처리 시나리오를 통해 구현한다.

03_Agent 간 메시지 전달 구조

결론적으로 Message Passing의 핵심은 다음 한 문장으로 정리할 수 있다.

"Agent가 다른 Agent의 상태를 직접 변경하는 것이 아니라, 명시적인 Message를 통해 필요한 작업과 결과를 전달하는 구조이다."

출처